



curso.dev

Aulas /web



Fórum

31 808



Em resposta a **Configurar o "baseUrl" para "Absolute Imports"**

marini 3 anos atrás

36 O meu já estava funcionando com `import database from "/infra/database.js"`; mesmo sem o arquivo `jsconfig.json`. Única diferença do meu caminho pro caminho do vídeo é que o meu caminho começava com uma `/`. Tinha feito essa alteração numa aula que o Filipe tinha perguntado...

- Será que se remover esses `"../..../../"` a aplicação vai quebrar?

ai eu testei e não quebrou, por isso eu acabei deixando assim, mas pelo que eu entendi no vídeo, não era esse o comportamento esperado. Será que não precisa mais do arquivo de config do javascript json?

Responder

FalaTuJean 1 ano atrás

1 Só passando para deixar meus parabéns a todos que compartilharam conhecimento aqui, foi muito conhecimento adquirido em uma seção de comentários, vocês são sensacionais galera!

Responder

AbnerCezar 1 ano atrás

14 Importações relativas x absolutas desempenho

Característica	Importações Relativas	Importações Absolutas
Complexidade de escrita	Maior, especialmente em projetos grandes	Menor, mais direta
Flexibilidade	Menos flexível para reestruturação de projeto	Mais flexível para mudanças na estrutura
Desempenho da compilação	Pode ser ligeiramente mais lento em projetos grandes	Potencialmente mais rápido devido à resolução direta

Característica	Importações Relativas	Importações Absolutas
Clareza para desenvolvedores	Pode ser confusa em projetos complexos	Mais clara e intuitiva
Manutenção	Requer atualizações frequentes ao mudar a estrutura	Menos propensas a erros de manutenção
Configuração adicional	Não requer configuração especial	Requer configuração do TypeScript ou Next.js
Exemplo de uso	<pre>import Component from '../../components/Component';</pre>	<pre>import Component from '/components/Component';</pre> OU <pre>import Component from '@components/Component';</pre>

Esta tabela mostra os comparativos entre importações relativas e absolutas em projetos Next.js. Ela destaca as principais vantagens e desafios de cada abordagem, tornando fácil para os desenvolvedores avaliarem as opções disponíveis ao planejar a estrutura de seus projetos.

Considerações adicionais:

O Next.js suporta aliases de módulos desde a versão 12.2, oferecendo uma solução ainda mais limpa para importações absolutas.

Usar aliases de módulos (`@components/*`) pode simplificar ainda mais as importações e melhorar o desempenho da compilação.

Para implementar importações absolutas, é necessário configurar o arquivo `jsconfig.json` ou `tsconfig.json`.

Em aplicações Next.js modernas, muitas vezes não é necessário um arquivo `jsconfig.json` se você está usando as configurações padrão do Next.js e seu projeto segue uma estrutura típica.

Caminhos absolutos como `/infra/database.js` geralmente funcionam bem no Next.js sem necessidade de configurações adicionais.

Esta tabela e considerações fornecem uma visão comparativa dos diferentes métodos de importação em projetos Next.js, ajudando a desenvolvedores a tomar decisões informadas sobre como estruturar seus projetos e suas importações.

Responder

^ [FernandoJose](#) 3 anos atrás

5 Aconteceu comigo também, mas resolvi aqui. Explicação muito boa.

Responder

^ IanFS 2 anos atrás

1 Excelente explicação. Aqui aconteceu a mesma coisa!

Responder

^ andreacruzmedes 3 anos atrás

103 Quando `/` não está presente no início, o caminho é interpretado como `relativo`, tomando o diretório do arquivo atual como ponto de partida. Porém, começando com `/` indicamos que o caminho é `absoluto` e aí a história muda:

O `Node.js` resolve caminhos absolutos a partir da raiz do sistema de arquivos do sistema operacional, porém o `Next.js`, quando executado pelo comando `npm run dev`, considera o diretório de trabalho (no terminal) como sendo o ponto de partida, e geralmente essa é a raiz do projeto, por isso funcionou.

Veja [essa explicação do Bing](#):

Considerando que `baseUrl` não está configurado, como o `npm run dev` de um projeto `next.js` resolve durante as importações os caminhos absolutos de arquivos que começam com `"/`? Será considerado o diretório de trabalho, a raiz do projeto ou a raiz do sistema de arquivos do sistema operacional?

No `Next.js`, os caminhos absolutos de arquivos que começam com `"/` são resolvidos com base no diretório de trabalho atual, que geralmente é a raiz do projeto. Portanto, se o `baseUrl` não estiver configurado, o `Next.js` considerará a raiz do projeto como o ponto de partida para resolver esses caminhos. Por exemplo, se você estiver executando o comando `npm run dev` na raiz do seu projeto, o `Next.js` procurará os módulos ou arquivos importados a partir desse local. Portanto, é importante garantir que você esteja no diretório correto ao executar comandos `npm`.

A propósito, o comando `npm` pode alterar o diretório de trabalho se passarmos o parâmetro `--prefix`.

Além disso, não devemos esperar o mesmo comportamento no ambiente de produção. Por isso, configurar `baseUrl` como `"."` no `jsconfig.json` faz todo caminho ser resolvido assumindo como ponto de partida sempre a raiz do projeto. Essa configuração é muito importante para remover as dúvidas que podemos ter sobre como o ambiente deve se comportar diante das referências nas importações, e essa certeza traz mais segurança para o desenvolvedor saber o que está fazendo.

Responder

^ GabrielFTeles 4 meses atrás

- 2 Essa com toda certeza foi uma das melhores discussões e explicações sobre assuntos relacionados a aula, que eu acompanhei até agora.

Todo mundo se ajudando e apesar de ter chegado em algo que parece satisfatório, ninguém se deu por vencido por uma explicação que ainda não tinha nada oficial batendo o martelo. Foi muito legal ver que quando essa discussão se iniciou vocês buscaram o máximo de informação possível, tentaram de certa forma ver como tudo funcionava debaixo dos panos e relataram aqui, tanto o que descobriram, quanto como estavam buscando estas informações.

Esse curso me surpreende cada vez mais, porque ele deixa claro que para sermos cada vez mais seniores, precisamos buscar a fundo e entender sobre, não basta ver que funcionou e bola pra frente, fico muito feliz de estar participando agora e uma pena não ter tido a oportunidade de quebrar a cabeça com vocês quando se iniciou esta discussão. Mas fico extremamente feliz de poder colher os frutos e conhecimentos compartilhados aqui, valeu turma!

Responder

^ andrecruzmendes 4 meses atrás

- 1 Sim! A turma aqui do curso é sensacional. Já faz praticamente dois anos, eu nem lembrava mais 😊 foi legal rever. Obrigado por comentar.

Responder

^ NatanRincker 2 anos atrás

- 2 Obrigado pela explicação detalhada sobre o comportamento do Next.js e da importância de configurarmos o baseUrl

Responder

^ Robinson 2 anos atrás

- 4 Brow que informações valiosas. Vcs vão nos incentivando (os sem experiência como eu) a buscar cada vez mais conhecimento, e o interessante é que de tanto vir essas interações as percepções dos problemas e suas resoluções vão desenhando mais fácil na cabeça. Viva a plasticidade neuronal!!!!

Responder

^ andrecruzmendes 2 anos atrás

- 1 Fantástico Robinson ! Essa seção de comentários é realmente sensacional. Sempre tem alguém mais experiente para iluminar o nosso processo de aprendizagem, e

sempre tem alguém menos experiente que nós podemos apoiar de alguma forma também. Então, a ideia é exatamente essa, aproveitando que esse ambiente é super saudável, é sempre positivo compartilhar nos comentários uma ideia, um pensamento, uma experiência, uma dúvida... O importante é essa interação que de fato nos motiva a continuar aprendendo.

Responder

^ Robinson 2 anos atrás

1 Verdade André ! Isso que nos leva adiante. Grato pela assertividade.

Responder

^ TucaSRL 2 anos atrás

2 3 coins, parabéns!

Responder

^ andrecruzmentes 2 anos atrás

1 😊 Obrigado.

Responder

^ filipedeschamps **instrutor** 3 anos atrás

5 Sensacionaaaaal todas estas informações meu caro!! E concordo então em manter o `baseUr1` para normalizar esta informação por todos os ambientes (e também ajudar no IntelliSense) 🍌

De qualquer forma, independente da explicação do Big, eu não consegui ainda encontrar nada oficial na documentação oficial do Next.js sobre esse comportamento. Quando o assunto é `Absolute Imports` , só consigo encontrar aquela documentação que mostrei na aula. Onde será que está registado esse comportamento?

Responder

^ andrecruzmentes 3 anos atrás

6 Então, confesso que também estou frustrado de não encontrar uma doc oficial. As referências que o Bing utilizou são essas: 1, 2, 3 e 4. Nenhuma é tão cirúrgica sobre esse assunto, ainda estou com uma pulga atrás da orelha.

Acho que seria interessante investigar como o `npm run dev` invoca o `node.js` por baixo dos panos, se por padrão o parâmetro `--prefix` assume o diretório de trabalho do terminal e se isso afeta as importações.

De todo modo, não custa nada fazer um teste direto no ambiente de produção, removendo o arquivo `jsconfig.json` e mantendo a importação como `/infra/database.js`, se possível até printando como esse caminho foi resolvido. Se eu conseguir fazer isso, mais tarde volto para compartilhar o resultado.

Responder

^ [filipedeschamps](#) **instrutor** 3 anos atrás

2 Showwww meu caro!!! Muito obrigado por nos ajudar nessa 🥰 🍌 🍌

Responder

^ [andreacruzmedes](#) 3 anos atrás

12 Filipe, testei em produção importando `/infra/database.js` e sem o arquivo `jsconfig.json`.

Como ainda não temos o banco, então adicionei uma simples função de teste no arquivo `database.js`:

```
function teste() {  
  return 'ok';  
}
```

```
export default {  
  teste: teste,  
};
```

E o arquivo `index.js` dentro de `status` ficou assim:

```
import database from "/infra/database.js";
```

```
async function status(request, response) {  
  response.status(200).json({  
    teste: database.teste(),  
  });  
}
```

```
export default status;
```

Fiz o `push`, esperei o deploy na Vercel e executei a rota no browser.

... E FUNCIONOU!!

Curioso, busquei ainda mais informações sobre isso porém não encontrei nada substancial. A única informação interessante que encontrei foi gerada pelo chat GPT, fundamentado por Ryan Dahl, Isaac Z. Schlueter, Guillermo Rauch, Naoyuki Kanezawa, Tony Kovanen e Dan Zajdband sobre node.js, npm e next.js. Ele disse o seguinte:

O Next.js usa uma configuração chamada `baseUrl` para lidar com esses caminhos absolutos. No entanto, se o `baseUrl` não estiver configurado, ele assume que a raiz do projeto é o ponto de partida para os caminhos absolutos.

Se não há configuração de `baseUrl` e não há plugins específicos para configurar imports absolutos, o Next.js usa o diretório de trabalho (o diretório de onde o comando `npm run dev` foi executado) como base para resolver esses caminhos absolutos.

Perguntei então como o next.js sabe qual é a raiz do projeto, e ele respondeu:

O Next.js utiliza algumas heurísticas para determinar automaticamente a raiz do projeto quando o arquivo `jsconfig.json` não está presente e não há plugins específicos configurados. O comportamento padrão é baseado no diretório onde o arquivo `package.json` do projeto está localizado.

Quando você executa o comando `npm run dev` ou `yarn dev` para iniciar o servidor de desenvolvimento do Next.js, o Next.js procura pelo arquivo `package.json` no diretório de trabalho atual. Ele considera o diretório onde encontra o `package.json` como a raiz do projeto. Portanto, esse diretório é usado como ponto de partida para resolver caminhos absolutos que começam com `"/`.

Então, se não houver um arquivo `jsconfig.json` e nenhum plugin configurado, o Next.js confia no diretório do `package.json` para determinar a raiz do projeto e resolver caminhos absolutos a partir desse ponto.

Pedi alguma referência sobre o assunto, e ele:

As informações que forneci anteriormente são baseadas em conhecimentos adquiridos até janeiro de 2022 e na documentação oficial do Next.js disponível naquele período.

Pedi ainda algumas palavras chave para ajudar na busca sobre o tema, uma delas me chamou a atenção:

Automatic Configuration: Buscar por seções relacionadas à configuração automática ou heurísticas usadas pelo Next.js para determinar configurações padrão na ausência de configurações explícitas.

Até a minha última atualização de conhecimento em janeiro de 2022, o termo "Automatic Configuration" no contexto do Next.js geralmente se refere à capacidade do framework de determinar automaticamente certas configurações com base nas convenções e padrões comuns, proporcionando uma experiência de desenvolvimento mais suave sem a necessidade de muita configuração manual.

Alguns aspectos da "Automatic Configuration" no Next.js incluem:

Determinação da Raiz do Projeto: Quando nenhum arquivo `jsconfig.json` está presente e nenhum `baseUrl` é configurado explicitamente, o Next.js utiliza heurísticas para determinar a raiz do projeto com base no diretório onde o arquivo `package.json` está localizado.

Se quiser, você pode ver a conversa completa [aqui](#).

Em síntese

Me parece que o `next.js` assume a localização do arquivo `package.json` como sendo a raiz do projeto e utiliza esse caminho em `absolute imports`. Mas ainda sinto falta de uma documentação oficial sobre isso.

Responder

- ^

liseu

2 anos atrás
- 1

Depois de retirar todos os `"/` do path e o teste falhar, minha primeira atitude foi colocar o `"/` no início, assim com fez `"/infra/database.js`. Mas com o @Filipe disse que ainda teria mais conteúdos a respeito eu não comentaria nada por aqui, por enquanto. Contudo, ao ler seu comentário/pesquisa, também fiquei curioso e olhar o código, mas vi que o @fhmurakami já tinha feito isso. Por curiosidade fui assuntar com o chatgpt e ele ainda

desconhece alegando sua atualização é de janeiro de 2022.
Concluiu dizendo:

"O Next.js Canary (ou "canário") refere-se à versão de desenvolvimento em constante evolução do Next.js. Essa versão pode incluir recursos experimentais ou alterações que ainda não foram totalmente testadas ou lançadas em uma versão estável.

Se o Next.js Canary inclui um arquivo tsconfig.json na sua estrutura de projeto, isso pode ser parte da configuração padrão ou experimental que a equipe do Next.js está testando. O arquivo tsconfig.json é usado para configurar as opções do TypeScript no projeto."

Responder

^ [filipedeschamps](#) **instrutor** 3 anos atrás

1 Que pesquisa sensacionaaaaa! meu caro 🥰🥰🥰🥰 muito massa mesmo!!!

E de fato, o que falta agora é alguma documentação oficial do que está acontecendo 🧡 Incluindo desde quando esse comportamento foi habilitado (qual versão do Next.js) 🙌

Responder

^ [fhmurakami](#) 2 anos atrás

1 Fui olhar no código do Next.js e tem um tsconfig.json onde eles configuram o baseUrl para . .

<https://github.com/vercel/next.js/blob/canary/tsconfig.json>

Responder

^ [filipedeschamps](#) **instrutor** 2 anos atrás

1 Massa fhmurakami ! Mas o mistério funciona de onde está na documentação o comportamento do / 🧡

Responder

^ [fhmurakami](#) 2 anos atrás

3



Fiz as mesmas perguntas que o andrecruzmentes no ChatGPT, mas obtive uma resposta um pouco diferente, diz que o Next.js assume o comportamento padrão do Node.js se não for especificado o baseUrl:

Até a minha última atualização em janeiro de 2022, o Next.js não possui uma resolução de caminhos absolutos incorporada de forma nativa como parte da sua configuração padrão. No entanto, muitos projetos Next.js optam por usar soluções de terceiros, como babel-plugin-root-import ou next-plugin-absolute-imports, para lidar com caminhos absolutos.

Se, no seu projeto, o baseUrl não está configurado e você não está usando nenhum plugin específico para tratamento de caminhos absolutos, o comportamento padrão do Node.js e do sistema de módulos do ECMAScript será aplicado.

Quando um caminho absoluto (iniciando com "/") é usado em uma importação no Node.js, ele é tratado como um caminho relativo à raiz do sistema de arquivos. Isso significa que o Node.js tentará importar o módulo ou arquivo começando do diretório raiz do sistema de arquivos.

Portanto, o comportamento padrão seria considerar a raiz do sistema de arquivos do sistema operacional. Isso está em conformidade com a maneira como os caminhos absolutos são normalmente tratados no Node.js quando não há configurações específicas no projeto para alterar esse comportamento.

No Next.js, quando não há um arquivo `jsconfig.json` e nenhum plugin específico configurado para resolver caminhos absolutos, o comportamento padrão do Node.js e do Next.js é aplicado. O Node.js resolve caminhos

absolutos iniciando do diretório raiz do sistema de arquivos.

No contexto do Next.js, isso significa que, ao importar um módulo ou arquivo com um caminho absoluto que começa com "/", o Next.js procurará esse módulo ou arquivo a partir da raiz do sistema de arquivos.

Por exemplo, se você tem uma estrutura de projeto assim:

```
/pages
  /subfolder
    Page.js
/components
  Component.js
```



Se você está em `Page.js` e deseja importar `Component.js`, você pode fazer algo assim:

```
import Component from '/components/Component.js'
```



O Next.js, usando a resolução padrão do Node.js, procurará `Component.js` a partir da raiz do sistema de arquivos.

Lembre-se de que essa é uma convenção padrão do Node.js e pode ser influenciada pelo ambiente em que seu aplicativo Next.js está sendo executado. Certifique-se de consultar a documentação mais recente do Next.js para quaisquer atualizações ou alterações no comportamento padrão.

Mas também não consegui encontrar essa informação na documentação oficial do Next.js

Responder

^ Lince 1 ano atrás

2 É difícil considerar as sugestões de AI para questões factuais específicas, especialmente

aquelas pouco documentadas já que, as Als fornecendo suas respostas com base na probabilidade dessas palavras estarem juntas conforme observado na web, receberam pouquíssimos dados sobre o objeto sendo questionando de modo que sua resposta não tem grande probabilidade de derivar de uma fonte real desse conhecimento - já que existem poucas - a não ser que você consiga fazer um prompt cirurgico. *(Talvez não se deva levar minha resposta tão a sério, já que não sou especialista em AI, mas é o que parece dado minha compreensão limitada)*

O que posso trazer sobre o comportamento padrão do node é que na situação proposta pelo ChatGPT não observa-se o comportamento esperado. Ao utilizar o package `path` para resolver um path absoluto - assim como o chatGPT falou, a root se dá na root do OS - entretanto diferente do que sugeri o chatGPT, isso não permite que você utilize paths absolutos para importar modulos a partir da pasta de seu projeto.

```
import {path} from "path"
console.log(
  path.resolve("/components/Component")
);
// Saída esperada em Windows: "C:\component
```



Naturalmente, a pasta de seu projeto não é a root do OS - no caso do windows `C:\` - portanto, a solução do GPT não funciona e não é adequada.

Dessa forma, retoma-se a sugestão dos colegas de ser um funcionalidade - potencialmente não documentada - do `Next.js` .

Responder

^ [LucasFS](#) 2 anos atrás

- 1 Depois de ver falhar sem a barra eu também fiz esse teste da `/` e me assustei ao ver passar. Mas depois eu testei remover a barra novamente e... não falhou. A partir daí, eu podia colocar o que eu quisesse na string do import, nunca falhava... tive que parar de rodar a aplicação e reiniciar para voltar a falhar, deve ter a ver com algum bug de cache igual o [Filipe](#) comentou em resposta a comentários parecidos em aulas anteriores.

Acreditei que por conta disso, o caso de teste com a `/` só não havia acusado erro devido ao bug, mas, pelas respostas de vocês, não, realmente funciona. Mais um exemplo de choque por não testar apropriadamente todos os estados kkkk

Responder

^ [yuriperim](#) 8 meses atrás

- 1 Cara, também encontrei uns comportamentos estranhos. O teste passava, mas o servidor acusava que não conseguia resolver o import, mas mesmo assim vinha o resultado da query. E não era consistente esse comportamento, porque as vezes passava com esse aviso, e as vezes realmente não passava. Fez eu até duvidar se `'infra/database.js'` realmente passava.

Responder

^ [crfelipe](#) 4 meses atrás

- 1 estou usando a versão mais nova do next a 16, e o `/` não funciona, olhado a documentação ele fala para usar o `"@"`, que também não funcionou, mas uma pesquisa entendi que o `@` é usado quando

